Salient Object Detection – ML Project

Author: Ed Haliti

genpact

Table of contents

Introduction

Salient Object Detection (SOD) is the task of identifying and segmenting the most visually prominent region(s) in a scene, the parts that naturally capture human attention. Unlike generic object detection, SOD produces a pixel-level binary mask highlighting the dominant subject rather than labelled bounding boxes.

This project implements a complete deep-learning SOD pipeline entirely from scratch using PyTorch, covering dataset preparation, a custom CNN encoder-decoder, combined loss function, training loop with checkpointing, quantitative evaluation, and an interactive Gradio demo.

Motivation

SOD is a foundational component in image captioning, video surveillance, content-aware image resizing, and autonomous driving perception. By building every component manually, without pretrained backbones, the project reinforces core ML concepts: convolution arithmetic, gradient flow, regularisation, and metric interpretation.

Objectives

✓ Design and train a CNN encoder-decoder from scratch
✓ Implement a combined BCE + IoU loss function.
✓ Build a complete data pipeline with augmentation
✓ Evaluate using IoU, Precision, Recall, F1, and MAE
✓ Compare a baseline model against an improved (U-Net style) variant
✓ Deliver an interactive demo showcasing real-time inference

Dataset & Preprocessing

Dataset Used: ECSSD

The system is dataset-agnostic. Any SOD dataset with a matching images/ and masks/ folder structure is supported. For reproducible testing, ECSSD (1,000 semantically meaningful images) with circular saliency masks was generated.

Preprocessing Steps

Step	Operation	Detail
1	Load	PIL Image => RGB conversion
2	Resize	Bilinear interpolation to 128 × 128 (or 224 × 224)
3	Normalise	ImageNet mean/std: μ=[0.485,0.456,0.406], σ=[0.229,0.224,0.225
4	Binarise	Mask threshold at 0.5 → float32 {0, 1}
5	Split	70 % train / 15 % val / 15 % test (seed = 42)

Data Augmentation

Augmentation	Operation
Horizontal flip	p = 0.5
Vertical flip	p = 0.3
Random crop + resize	Crop to 80 % of size, then resize back; p = 0.5
Random rotation	±15°; p = 0.5
Brightness jitter	Factor ∈ [0.7, 1.3]; p = 0.5
Contrast jitter	Factor ∈ [0.7, 1.3]; p = 0.5
Saturation jitter	Factor ∈ [0.7, 1.3]; p = 0.5

All spatial augmentations (flip, crop, rotation) are applied identically to both the image and its mask to preserve pixel-level alignment. Colour augmentations are applied to images only.

Model Architecture

Baseline CNN Encoder-Decoder

The baseline architecture follows a symmetric encoder-decoder pattern. The encoder progressively downsamples the feature map through convolutional blocks followed by max-pooling, building a rich high-level representation. The decoder upsamples via transposed convolutions and predicts a binary mask at the original resolution.

Component	**Purpose**
Encoder	Extract high-level features
Bottleneck	Deep representation
Decoder	Upsample features
Output	1-channel sigmoid segmentation mask

Improved Model — BatchNorm + Dropout + Skip Connections

The improved variant introduces three architectural changes motivated by known best practices:

• BatchNorm2d after every Conv2d layer, stabilises activations, allows higher learning rates, and reduces internal covariate shift.

• Dropout2d (p=0.2) in the bottleneck, regularises the most parameter-dense region, reducing overfitting on small datasets.

• Skip connections (U-Net style), encoder feature maps are concatenated with the corresponding decoder level, preserving fine spatial detail lost during downsampling. This is the single largest contributor to improved IoU.

The improved model has ~4× more parameters than the baseline due to deeper channels (64→128→256→512→1024) but converges faster and generalises better thanks to the skip connections and normalisation.

Loss Function

A composite loss combines binary cross-entropy (BCE) and IoU loss:

$$L = \mathrm{BCE}(\ , y) + 0.5 \times (1 - \mathrm{IoU}(\ , y))$$

BCE provides dense pixel-level supervision while the IoU term directly optimises the overlap metric. The 0.5 weighting prevents the IoU component from dominating early training when predictions are near-random.

Training & Configuration

Hyperparameter	Value
Optimiser	Adam (β =0.9, β =0.999, ε=1e-8)
Learning rate	1e-3 with ReduceLROnPlateau (factor=0.5, patience=3)
Max epochs	25
Early stopping	patience = 5 epochs on val loss
Batch size	16
Gradient clipping	max_norm = 1.0
Seed	42

Training Loop

Each epoch executes: (1) forward pass through the model, (2) loss computation, (3) backward pass via automatic differentiation, (4) gradient clipping, (5) parameter update. After each epoch, the validation loss and metrics are logged to CSV and printed to the console.

Checkpoint System (Bonus)

After every epoch a checkpoint is saved containing the model weights, optimiser state, scheduler state, and current epoch index. On restart, the training script automatically detects the latest checkpoint and resumes from that point, ensuring no epochs are wasted if a Colab session times out or a power interruption occurs.

Early Stopping

If the validation loss does not improve for 5 consecutive epochs, training terminates and the weights from the best epoch are preserved. This prevents overfitting and reduces unnecessary compute.

Results & Evaluation

Evaluation was done using random test samples.

Metric	**Formula**	**Interpretation**
Precision	TP / (TP + FP)	Fraction of predicted positives correct
IoU	TP / (TP + FP + FN)	Overlap between pred. and GT mask
Recall	TP / (TP + FN)	Fraction of true positives detected
F1 Score	2 × P × R / (P + R)	Harmonic mean of precision & recall
MAE	mean $\lvert \ - y \rvert$	Pixel-level absolute error
Accuracy	(TP+TN) / (TP+FP+FN+TN)	Overall pixel classification accuracy

Results

Model	Baseline	Improved
Precision	0.79	0.86
IoU	0.72	0.81
Recall	0.84	0.89
F1 Score	0.81	0.87
MAE	0.08	0.05
Accuracy	0.88	0.93

Key Learnings

This project successfully implemented a full Salient Object Detection pipeline from scratch, achieving solid quantitative results and demonstrating several key deep learning insights

Main Things I Learned

- Combined losses converge faster. Adding an IoU term to pure BCE provides a direct gradient signal aligned with the evaluation metric.
- Skip connections are transformative. Concatenating encoder features into the decoder recovers boundary precision that pooling discards, the single most impactful change across all experiments
- Checkpoint resumption is essential. In cloud environments (Colab, Kaggle) session timeouts are common; robust checkpoint logic eliminates lost progress.
- Augmentation quality matters. Spatial augmentations must be applied identically to image and mask; inconsistency causes the model to learn artefactual correlations.
- IoU is the right metric. Accuracy is misleading on unbalanced masks; IoU and F1 reveal real-world performance far more reliably.

Soft Skills Learned

- How to read errors and debug step-by-step
- How to research solutions instead of copying exact code
- How to properly document a machine learning project
- Importance of version control (Git & GitHub) for safe backups

This project helped me understand how real AI projects are built, not just theory but also structure, coding habits, and presentation.

Final Statement

This was the most complete ML project I have done so far. At the start it was confusing, but working piece by piece showed me how deep learning really works. I now have more confidence to build models, analyze results, and present AI projects professionally.

Thank you

For further information, please contact:

Ed Haliti

AI Engineer - Intern

T +3834465092

E ed.haliti @genpact.com

W www.genpact.com

genpact